DATAMITES

in association with Rubiex Internship Program

ARTIFICIAL INTELLIGENCE CAPSTONE PROJECT

PRAICP-1002 : TrafSignDetc — Traffic Sign Recognition

*A Report Submitted in Partial Fulfilment of the Requirements of the **Artificial Intelligence Capstone Project***

Project Title	PRAICP-1002: TrafSignDetc — Traffic Sign Recognition
Project Code	PRAICP-1002
Team Code	PTID-AIE-JUL-26-11142
Project Type	Artificial Intelligence Capstone Project
Dataset Source	DataMites Capstone Project Dataset
Dataset Name	TrafSignDetc (directories of traffic sign images, one directory per sign)
Submitted By	Sameeksha
Organisation	DataMites / Rubiex Internship Project
Report Date	October 2026

DataMites® Innovative Solutions Pvt. Ltd.
Bengaluru, Karnataka, India

About the Author / Candidate Profile

This section presents a brief profile of the candidate who has authored and submitted this capstone project report, along with the administrative details under which the project was carried out as part of the DataMites Artificial Intelligence Engineer program delivered in association with the Rubiex Internship initiative.

Candidate Name	Sameeksha
Program	Artificial Intelligence Engineer (AIE) — DataMites
Internship Track	Rubiex Internship Project
Team Code	PTID-AIE-JUL-26-11142
Project Code	PRAICP-1002
Project Title	PRAICP-1002: TrafSignDetc — Traffic Sign Recognition
Project Type	Artificial Intelligence Capstone Project
Domain	Intelligent Transportation / Computer Vision — Deep Learning (Image Classification)
Tools & Technologies	Python, TensorFlow, Keras, Keras Tuner, NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, Pillow
GitHub Repository	https://github.com/Sameekshagithub/PRAICP-1002-Traffic-Sign-Detection-Using-Deep-Learning.git
Supporting Drive Link	https://drive.google.com/drive/folders/1pcADNaZ2l3gGNx7uDhTEBlcqMi5VIEOh?usp=drive_link
Report Prepared On	October 2026

A Note on this Report

This document has been compiled as a comprehensive, stand-alone project report derived from the candidate's Jupyter Notebook submission (PRAICP_1002_TrafSignDetection.ipynb) for the PRAICP-1002 capstone assignment. It restates the problem statement, methodology, code-level design decisions, exploratory findings, model architecture, hyperparameter tuning results, quantitative results and business conclusions of the original notebook in a structured, examiner-friendly format suitable for academic evaluation and portfolio presentation. All figures, tables and numerical results quoted in this report are taken from the executed outputs of the notebook.

Declaration

I, Sameeksha, hereby declare that this project report titled “PRAICP-1002: TrafSignDetc — Traffic Sign Recognition” is a record of an original piece of work carried out by me as part of the Artificial Intelligence Capstone Project (Project Code: PRAICP-1002, Team Code: PTID-AIE-JUL-26-11142) under the DataMites Artificial Intelligence Engineer program, undertaken in association with the Rubiex Internship Project.

I further declare that the analysis, model development, results and conclusions presented in this report are based on my own work using the dataset supplied for this capstone project, and that this report has not been submitted, in whole or in part, for the award of any other certification or qualification.

All external libraries, tools (Keras Tuner) and reference material used in the course of this project have been duly acknowledged in the relevant sections of this report.

Signature: _____

Name: Sameeksha

Date: October 2026

Acknowledgement

I would like to express my sincere gratitude to DataMites® Innovative Solutions Pvt. Ltd. and the Rubiex Internship Project for providing me with the opportunity, mentorship, curriculum and dataset required to undertake this Artificial Intelligence Capstone Project on Traffic Sign Recognition.

I am grateful to my mentors and instructors for their continuous guidance on deep learning concepts, convolutional neural network design, hyperparameter tuning methodology and evaluation techniques for multi-class image classification problems, which greatly enriched my understanding of applying Artificial Intelligence to real-world road-safety challenges.

I also wish to thank my peers and the DataMites support team for their valuable feedback during the course of this project, and my family for their constant encouragement throughout the duration of this internship and capstone submission.

Table of Contents

About the Author / Candidate Profile
Declaration
Acknowledgement
Executive Summary
1. Problem Statement & Project Objective
2. Import of Python Libraries & Tools
3. Dataset Upload & Domain Analysis
4. Basic Checks on the Dataset
5. Exploratory Data Analysis (EDA)
6. Image Preprocessing
7. Train / Validation / Test Split & Data Preparation
8. Data Augmentation
9. Model Building & Training — Baseline CNN
10. Hyperparameter Tuning
11. Final CNN Model (Tuned)
12. Model Comparison Report
13. Best Model — Detailed Evaluation
14. Predicting the Traffic Sign of a New Image
15. Saving the Model
16. Business Insights
17. Challenges Faced
18. Final Conclusion
19. Project Links
20. Ethical Considerations & Limitations
21. Future Scope & Recommendations
22. References
Appendix A: Glossary of Terms
Appendix B: Environment & Reproducibility Notes

Executive Summary

Traffic signs tell drivers what they must do, what they must not do and what dangers lie ahead. Missing or misreading a sign — a stop sign, a speed limit or a warning — is a common cause of road accidents, and driver-assistance systems and self-driving vehicles therefore need to recognise traffic signs automatically from camera images. This capstone project, undertaken as part of the DataMites Artificial Intelligence Engineer program in association with the Rubiex Internship Project, builds a Convolutional Neural Network (CNN) that looks at a photograph of a traffic sign and predicts which of 43 sign classes it belongs to.

The project uses the DataMites-supplied TrafSignDetc dataset, which contains 39,209 labelled images across 43 traffic sign classes (folders 0–42). The full dataset is strongly imbalanced (an imbalance ratio of 10.71 : 1 between the largest class, 2, with 2,250 images, and the smallest class, 0, with 210 images) and its images come in 2,066 different native resolutions, ranging from 25×25 up to 224×219 pixels. Because the full dataset is too large to hold comfortably in memory on ordinary hardware, a memory-safe strategy was adopted: the ZIP archive was indexed without extracting it, and a random sample of at most 400 images per class was extracted, giving a working set of 15,460 images (imbalance ratio 1.90 : 1). Every image was resized to 64×64 RGB, stored as compact 8-bit integers, and normalised inside the network by a Rescaling layer. The working set was divided into stratified training, validation and test sets of 10,822, 2,319 and 2,319 images, and class weights and data augmentation (rotation, shift, zoom and contrast, with horizontal flipping deliberately excluded) were used to counter the remaining imbalance and the risk of overfitting.

Two CNN models were built and compared. A baseline CNN (688,747 parameters) reached a validation accuracy of 99.14% and a test accuracy of 99.35% (macro F1-score 0.9940). Keras Tuner's RandomSearch then evaluated eight hyperparameter combinations in about 2 hours 26 minutes on CPU-only hardware, selecting the configuration with the highest validation accuracy (0.9918). The final tuned CNN (780,971 parameters) achieved a validation accuracy of 99.61% and a test accuracy of 99.53%, with macro Precision, Recall and F1-score all equal to 0.9957 on the 2,319-image held-out test set. Only 11 of 2,319 test images were misclassified, and these errors were concentrated in visually similar signs such as the different speed limits.

This report documents the complete workflow — from problem definition and domain analysis, through memory-safe data engineering, exploratory analysis, preprocessing, augmentation, baseline modelling, systematic hyperparameter tuning and rigorous final evaluation — and closes with business insights, a candid discussion of limitations (including why a random image-level split is likely to give optimistic results on this kind of dataset), ethical considerations for safety-related technology, and concrete recommendations for extending this sign classifier into a complete detection-plus-recognition system.

1. Problem Statement & Project Objective

1.1 Problem Statement

Traffic signs are the visual language of the road. They communicate rules (stop, yield, no entry), limits (speed limits, no passing) and hazards (slippery road, pedestrians, children crossing, road work) to every driver, and they are deliberately designed to be recognised at a glance. In practice, however, signs are missed or misread — because of distraction, fatigue, poor visibility, or because the sign is small, dirty, partly hidden or seen from an awkward angle — and a missed stop sign or speed-limit sign is a common contributing factor in road accidents.

Modern driver-assistance systems and self-driving vehicles cannot rely on a human to read signs; they must recognise them automatically from camera images, quickly and reliably. The objective of this project is to develop a Convolutional Neural Network (CNN) based machine learning model that can look at a photograph of a traffic sign from the TrafSignDetc dataset and automatically predict which traffic sign it is, helping to improve road safety.

1.2 Project Objective

The DataMites project brief for PRAICP-1002 defines two explicit goals, both of which are addressed in this project:

1. **Image Preprocessing** — convert raw, variable-resolution traffic sign photographs into a clean, consistent numerical format (resized, normalised and augmented) that a neural network can learn from effectively.
2. **ML Model to Predict the Traffic Sign** — build, systematically tune and rigorously evaluate a CNN that classifies each image into its correct traffic sign class.

To achieve these two goals, the project follows a structured, end-to-end workflow:

1. **Load and explore the dataset** — examining its classes, class balance, native image resolutions and sample images, in a memory-safe way because the full dataset is very large.
2. **Preprocess the images** (resize to a fixed size, normalise pixel values) and split them into stratified Training, Validation and Test sets.
3. **Apply data augmentation** (small rotations, shifts, zooms and contrast changes, deliberately excluding horizontal flips) to improve generalisation.
4. **Build a baseline CNN**, then systematically improve it using hyperparameter tuning (Keras Tuner) to obtain a final, tuned CNN.
5. **Evaluate both models** using Accuracy, macro-averaged Precision/Recall/F1-score, a Confusion Matrix and a full Classification Report.
6. **Demonstrate end-to-end prediction** on a brand-new image, and save the trained model for future reuse.

1.3 Scope of the Project

The scope of this capstone project covers the full applied machine learning lifecycle for a multi-class image classification problem: dataset acquisition and inspection, domain analysis of traffic signs, data cleaning, exploratory data analysis, preprocessing and augmentation, baseline model design, systematic hyperparameter search, final model training, rigorous comparative evaluation, single-image inference, model persistence, and the translation of technical results into business insights and future recommendations. The project deliberately restricts its scope to the classification of signs that have already been cropped out of a photograph; it does not attempt to locate signs within a full road scene (detection), to read signs in video, or to handle signs outside the 43 classes of the dataset. These are identified as directions for future work in Section 21.

2. Import of Python Libraries & Tools

Before any data can be loaded or modelled, the required Python libraries are imported. These fall into broad functional categories: general-purpose data handling, dataset acquisition, image processing, deep learning and hyperparameter tuning, and performance evaluation. Table 2.1 summarises the principal libraries used in this project. A fixed random seed (42) is set for Python, NumPy and TensorFlow so that data sampling, splitting and training are repeatable.

Library	Category	Purpose in this Project
os, re, json, gc, random, zipfile, posixpath, textwrap, warnings	Core Python / OS	File-system navigation, reading the dataset ZIP index without extracting it, natural sorting of class folders, saving class-name metadata, memory clean-up and reproducible sampling.
NumPy, Pandas	Data Handling	Numerical image arrays and tabular summaries of class counts, image dimensions and model results.
Matplotlib, Seaborn	Visualisation	Class-distribution plots, sample images, training curves, comparison charts, the confusion matrix heatmap and the misclassified-image gallery.
Pillow (PIL)	Image Processing	Opening, verifying, converting and resizing individual traffic sign images.
Requests, tqdm	Dataset Acquisition	Streaming download of the dataset ZIP from the DataMites CDN link, with progress bars for long-running loops.
TensorFlow / Keras	Deep Learning	Building, compiling, training and saving the baseline and tuned CNN models.
Keras preprocessing layers (Rescaling, RandomRotation, RandomTranslation, RandomZoom, RandomContrast)	Data Pipeline	In-model normalisation and data augmentation; augmentation is active only during training.
Keras Tuner (RandomSearch)	Hyperparameter Tuning	Automated, systematic search over CNN architecture and training hyperparameters.
Scikit-learn	Evaluation	Stratified train/validation/test splitting, class-weight computation, confusion matrix, classification report and precision/recall/F1 metrics.

Table 2.1: Principal Python libraries used in the project and their purpose.

TensorFlow/Keras was selected as the deep learning framework because it provides a mature, high-level API for constructing custom CNN architectures with built-in preprocessing layers, alongside Keras Tuner, a companion library that automates hyperparameter search directly against Keras models. Scikit-learn complements this by supplying the stratified splitting utilities (essential given the project's class imbalance) and the classification-evaluation toolkit used throughout Sections 12 and 13.

3. Dataset Upload & Domain Analysis

3.1 Dataset Acquisition and Memory-Safe Loading

The dataset used in this project is TrafSignDetc (directories of traffic sign images, one directory per sign), supplied as the official DataMites Capstone Project Dataset for the PRAICP-1002 capstone assignment and downloaded programmatically from the archive linked below.

https://d3ilbtxij3aepc.cloudfront.net/projects/AI-Capstone-Projects/PRAICP-1002-TrafSignDetc.zip

The archive contains one directory per traffic sign, each holding the photographs of that sign; the directory name therefore directly encodes the label (class) of every image inside it. The class directories were found inside a folder named “Train” within the archive.

The full dataset is very large (39,209 images). Extracting everything and loading it into memory as arrays could exhaust the RAM of a normal laptop or a free cloud notebook session. The notebook therefore follows a memory-safe strategy:

1. The ZIP is **downloaded once** by streaming, and the download is skipped automatically if the file is already present.
2. The ZIP is **read without extracting it** to find the class directories and count the images in each class; this gives the “full dataset” picture used in the EDA.
3. Only a **fixed number of randomly chosen images per class** (MAX_IMAGES_PER_CLASS = 400, sampled with the fixed seed) is extracted to disk. Choosing per class keeps every sign represented, and classes with fewer than 400 images are included in full.
4. Images are stored as compact **uint8 arrays at a small size** (64×64), so the whole working set fits comfortably in memory (190 MB instead of 760 MB for a float32 copy).

Setting MAX_IMAGES_PER_CLASS to None uses the complete dataset on machines with enough RAM, while a smaller value (for example 100) allows a quick dry-run.

3.2 Domain Analysis

Traffic signs are designed to be recognised quickly by humans, so they follow strict visual rules: the shape of the sign (circle, triangle, octagon, diamond), its border and background colour (red for prohibition or warning, blue for mandatory instructions, yellow for priority) and the pictogram or number inside it. These are spatial visual patterns, which makes this a natural candidate problem for a CNN, which learns edges, shapes, colours and symbols directly from pixels rather than relying on hand-crafted features.

Real-world traffic sign photographs are, however, not clean. They are taken from moving vehicles, so images can be blurred, very small, too dark or too bright, partly hidden, or photographed from an angle. The class folders are numbered 0 to 42, which matches the standard 43-class traffic sign numbering of the German Traffic Sign Recognition Benchmark (GTSRB); the notebook uses this numbering to attach readable names (for example, 14 = Stop and 13 = Yield) to the class folders.

Two domain-specific observations shape the project. First, several signs are very similar to each other — for example, the different speed-limit signs differ only in the number inside the circle — which is expected, a priori, to make those classes the hardest to separate. This expectation is investigated using the confusion matrix and per-class F1-scores in Section 13. Second, left/right-meaning signs (for example “Turn left ahead” versus “Turn right ahead”, or “Keep left” versus “Keep right”) are mirror images of each other, so mirroring an image must never be used as data augmentation (Section 8.2).

4. Basic Checks on the Dataset

Before any exploratory analysis or modelling is undertaken, a set of basic sanity checks is performed to understand the size, quality and class balance of the dataset. A structured table is first built with one row per extracted image, recording its class label, file name, full path, width, height and colour mode; this table becomes the foundation for every subsequent basic check and EDA plot. The class balance of both the full dataset (read from the ZIP index) and the working subset actually used is reported.

4.1 Dataset Size and Class List

The full dataset contains 39,209 images across 43 traffic sign classes (folders 0 to 42). The working subset extracted for this project contains 15,460 images. All working-set images are confirmed to be RGB PNG files.

Property	Value
Total images in the full dataset (ZIP)	39,209
Images in the working subset	15,460 (at most 400 per class)
Number of classes	43 (folders 0–42)
File format	.png (all working-set files)
Colour mode	RGB (all 15,460 images)

Table 4.1: High-level dataset composition.

4.2 Corrupt / Unreadable Image Check

Every one of the 15,460 working-set image files was opened and verified programmatically to detect files that are missing, truncated or otherwise unreadable. This check found zero corrupt or unreadable images, confirming that the dataset is clean and that every labelled file can be safely loaded for preprocessing and training without needing any exclusion or repair step.

4.3 Class-wise Image Counts and Imbalance

Counting images per class shows that the full dataset is strongly imbalanced: the smallest class (0, Speed limit 20 km/h) contains only 210 images, while the largest class (2, Speed limit 50 km/h) contains 2,250 images — an imbalance ratio of 10.71 : 1. Capping each class at 400 images reduces the working subset's ratio to 1.90 : 1 (smallest class 0 with 210 images, largest classes at the 400-image cap). Table 4.2 compares the two, and Table 4.3 lists the working-subset count for every class.

Measure	Full dataset (ZIP)	Working subset
Total images	39,209	15,460
Smallest class	0 → 210 images	0 → 210 images
Largest class	2 → 2,250 images	1 → 400 images (cap)
Imbalance ratio (max / min)	10.71 : 1	1.90 : 1

Table 4.2: Class balance of the full dataset versus the working subset.

Class	Sign Name	Images (subset)
0	Speed limit (20km/h)	210
1	Speed limit (30km/h)	400
2	Speed limit (50km/h)	400
3	Speed limit (60km/h)	400

Class	Sign Name	Images (subset)
4	Speed limit (70km/h)	400
5	Speed limit (80km/h)	400
6	End of speed limit (80km/h)	400
7	Speed limit (100km/h)	400
8	Speed limit (120km/h)	400
9	No passing	400
10	No passing for vehicles over 3.5 tons	400
11	Right-of-way at next intersection	400
12	Priority road	400
13	Yield	400
14	Stop	400
15	No vehicles	400
16	Vehicles over 3.5 tons prohibited	400
17	No entry	400
18	General caution	400
19	Dangerous curve to the left	210
20	Dangerous curve to the right	360
21	Double curve	330
22	Bumpy road	390
23	Slippery road	400
24	Road narrows on the right	270
25	Road work	400
26	Traffic signals	400
27	Pedestrians	240
28	Children crossing	400
29	Bicycles crossing	270
30	Beware of ice/snow	400
31	Wild animals crossing	400
32	End of all speed and passing limits	240
33	Turn right ahead	400
34	Turn left ahead	400
35	Ahead only	400
36	Go straight or right	390
37	Go straight or left	210
38	Keep right	400
39	Keep left	300
40	Roundabout mandatory	360

Class	Sign Name	Images (subset)
41	End of no passing	240
42	End of no passing by vehicles over 3.5 tons	240

Table 4.3: Image count per traffic sign class in the working subset (Total = 15,460; 28 classes at the 400-image cap).

This remaining imbalance (1.90 : 1) is mild, and is not severe enough to require aggressive resampling techniques such as oversampling or synthetic minority image generation. It is nevertheless addressed by two complementary countermeasures, detailed further in Section 7: stratified splitting, which ensures every split preserves the same relative class proportions as the working set, and class weighting, which assigns proportionally higher loss penalties to under-represented classes (such as 0, 19 and 37) during training.

4.4 Image Resolutions

Traffic sign photographs come in many different sizes. The working subset contains 2,066 different width×height combinations. Widths range from 25 to 224 pixels (median 44) and heights from 25 to 219 pixels (median 43), so the typical image is very small. Table 4.4 lists the ten most frequent resolutions.

Width × Height (pixels)	Number of Images
31 × 31	142
38 × 38	138
30 × 30	134
29 × 29	134
33 × 33	128
32 × 32	119
30 × 31	117
36 × 36	115
32 × 33	115
34 × 35	112

Table 4.4: The ten most frequent original image resolutions (of 2,066 distinct resolutions).

5. Exploratory Data Analysis (EDA)

Exploratory Data Analysis is performed to visually and statistically understand the dataset before committing to a specific CNN architecture. The EDA in this project covers the class distribution of both the full dataset and the working subset, the native resolutions of the original (pre-resize) images, one representative sample image from every one of the 43 signs, and a closer look at multiple images within single sign classes to assess within-class variation.

5.1 Class Distribution

A bar chart of image counts per class for the full dataset (Figure 5.1) shows the strong imbalance quantified in Section 4.3: a number of classes (the largest being class 2 with 2,250 images) have around 2,000 images or more, while many others (such as 0, 19, 27, 32, 37, 41 and 42) have only a few hundred (210–240) images. After capping at 400 images per class, the working subset (Figure 5.2) is far more balanced, with a small number of classes remaining below the cap.

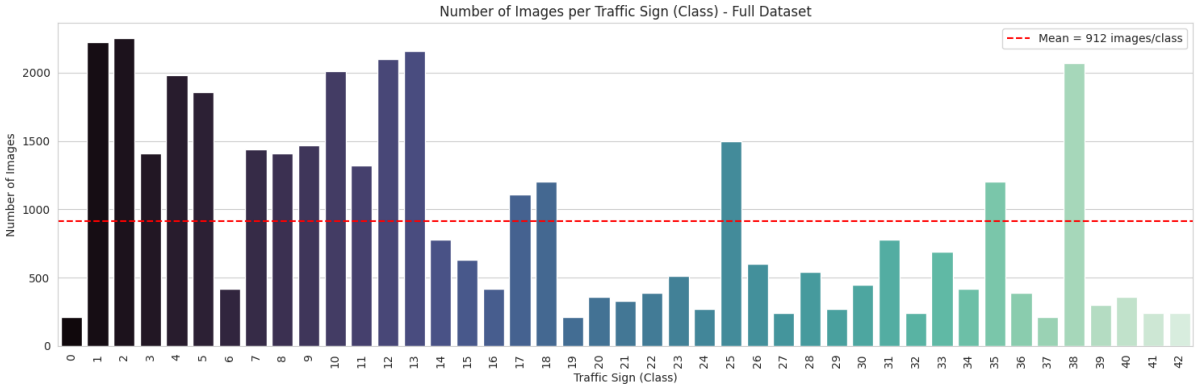


Figure 5.1: Number of images per traffic sign class — full dataset (dashed line = mean).

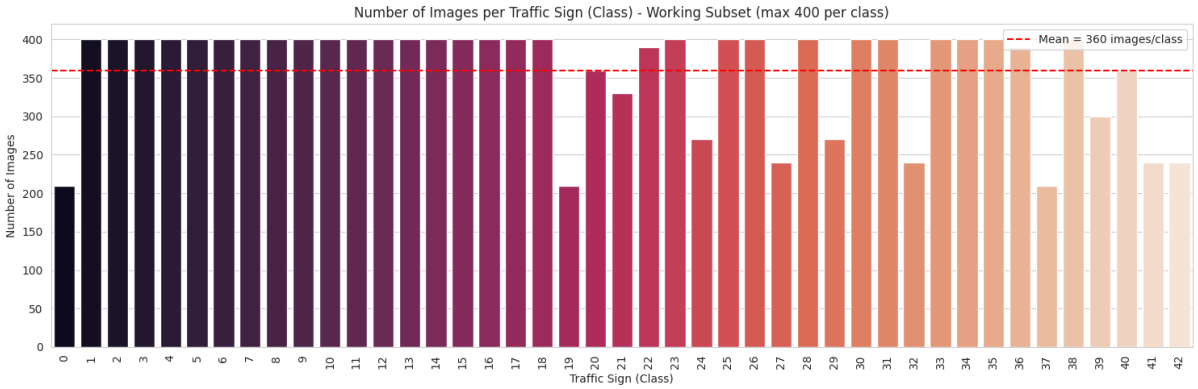


Figure 5.2: Number of images per traffic sign class — working subset (maximum 400 per class).

5.2 Original Image Resolutions

Unlike a dataset captured under one single, controlled protocol, these photographs vary enormously in size. The histograms in Figure 5.3 show that most images are small — the bulk of widths and heights lie between about 25 and 60 pixels — with a long tail of larger images. This mixture of resolutions confirms the necessity of the uniform resizing step performed in Section 6, since a CNN's convolutional and dense layers require fixed-size input tensors.

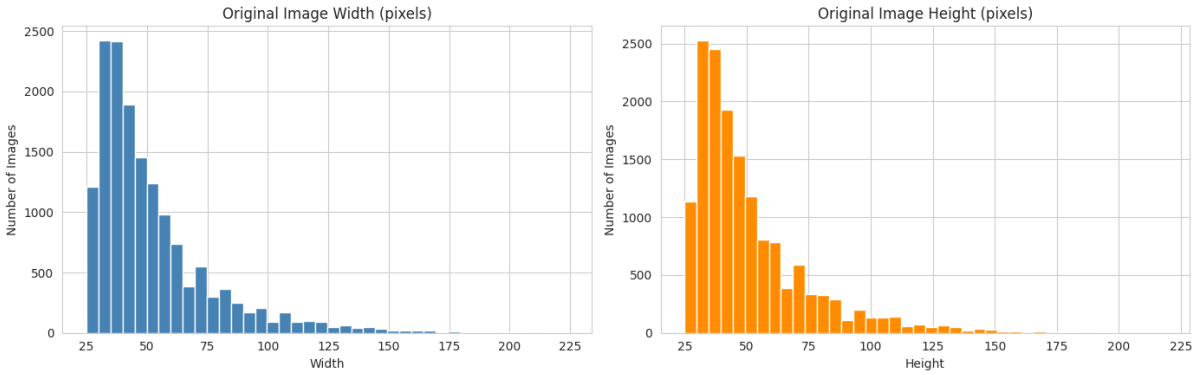


Figure 5.3: Distribution of original image widths and heights (pixels).

5.3 Sample Images: One per Sign

A grid displaying one representative sample image from each of the 43 sign classes (Figure 5.4) confirms the visual conventions described in Section 3.2 (shape, border colour and pictogram) and gives an initial, qualitative impression of how visually distinct — or similar — certain signs are. It also shows that many photographs are small, dark or low in contrast.

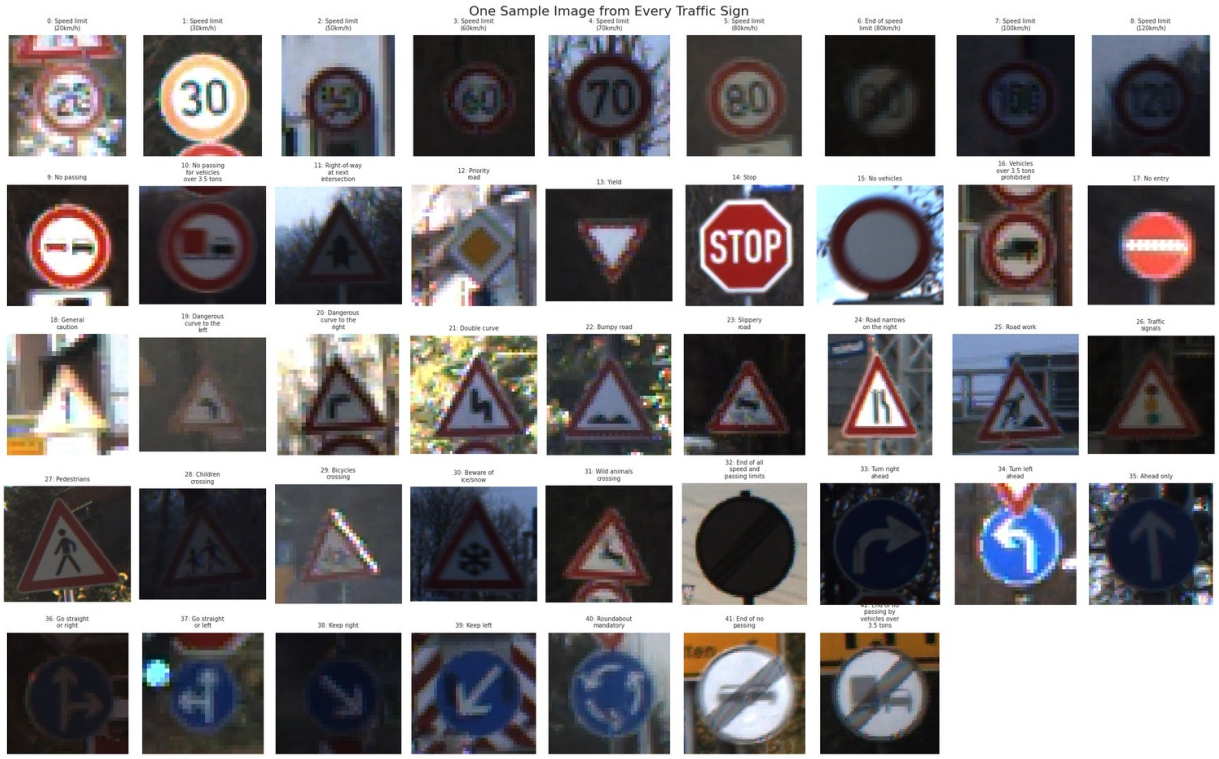


Figure 5.4: One sample image from every traffic sign class.

5.4 Within-Class Variation

To assess how much natural variation exists within a single class, ten random images of the same sign were plotted for two classes: sign 0 (Speed limit 20 km/h) and sign 21 (Double curve). Figures 5.5 and 5.6 show substantial variation in brightness, sharpness, size, viewing angle and background — some images are bright and crisp, others dark, blurred or tilted. This natural, real-world variability is an important justification for the data augmentation strategy adopted in Section 8, since a model that has only ever seen clean, centred images would struggle to generalise to the full diversity of real road photographs.



Figure 5.5: Ten random images of sign 0 (Speed limit 20 km/h).

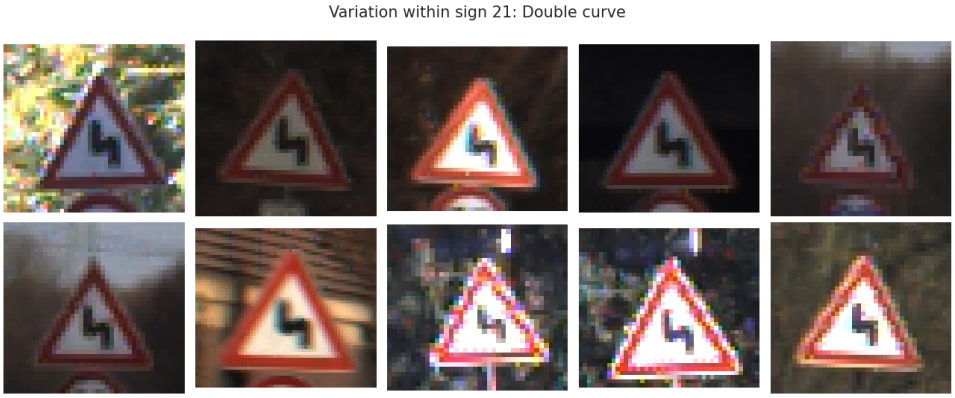


Figure 5.6: Ten random images of sign 21 (Double curve).

5.5 EDA Summary (Conclusion)

1. The dataset contains **43 traffic sign classes** and 39,209 images (15,460 used in the working subset), with one directory per sign. The check for corrupt or unreadable files found **none**.
2. The classes are **not equally sized**: the full dataset has an imbalance ratio of 10.7 : 1 (smallest class 0 with 210 images, largest class 2 with 2,250). This is addressed by capping images per class, **stratified splitting** and **class weights** during training.
3. The images come in **2,066 different resolutions** from 25×25 up to 224×219 pixels, and most are small. A CNN needs a fixed input size, confirming the need for the resizing step in preprocessing.
4. The sample images show that signs are photographed from vehicles: images within the same class vary in **brightness, sharpness, size, viewing angle and background**. This supports the use of **data augmentation** (small rotations, shifts, zoom and contrast changes).
5. Some signs are visually very close to each other (for example the different speed-limit signs, which differ only by the number inside the circle). These are expected to be the hardest to separate and are examined later in the confusion matrix (Section 13).

6. Image Preprocessing

Image preprocessing is the first of the two explicit goals defined in the DataMites project brief for PRAICP-1002. A CNN cannot directly accept images of differing sizes, and it learns most effectively when its numerical inputs sit on a small, consistent scale. Accordingly, two preprocessing steps are applied to every single image in the dataset, with no exceptions.

6.1 Resizing

Every image is first converted to RGB (guaranteeing exactly three colour channels regardless of its original mode) and then resized, using Lanczos resampling, to a fixed 64 × 64 pixel size, producing a consistent (64, 64, 3) array (height × width × Red/Green/Blue channels) for every image. Traffic signs are simple, high-contrast symbols, so 64 × 64 keeps the shape, colour and symbol clearly visible while keeping memory use and training time low. Since the median original image is only about 44 × 43 pixels, most images are in fact slightly enlarged rather than shrunk.

6.2 Normalisation

Raw pixel values naturally range from 0 to 255 (standard 8-bit colour depth). They are divided by 255 so that all inputs lie between 0 and 1, which makes gradient-based optimisation (as used by the Adam optimiser in this project) more stable and helps the network converge faster. To save memory, the resized arrays are stored as compact 8-bit integers (uint8, four times smaller than float32), and the division by 255 is performed by a Rescaling(1/255) layer at the very start of the CNN. This is mathematically identical to normalising the arrays beforehand, and it also guarantees that new images given to the model at prediction time are normalised in exactly the same way as the training images.

6.3 Consistency Between Training and Inference

A single, shared preprocess_image() function is defined once and reused everywhere an image needs to be prepared for the model — during initial dataset loading and later during prediction on a brand-new image in Section 14. Because normalisation lives inside the model itself, a new photograph is processed in exactly the same way as every training image (identical resize target, identical channel handling, identical normalisation). This is essential: if training and inference preprocessing ever diverged, the model would silently receive data that looks statistically different from what it learned on, degrading real-world accuracy.

6.4 A Note on Aspect Ratio

Some images are not perfectly square, so resizing every image to a fixed 64 × 64 square slightly changes the aspect ratio of those images. This trade-off was accepted deliberately: since the exact same transformation is applied uniformly to every training image and every future prediction image, the CNN simply learns from the resized versions, which is internally consistent.

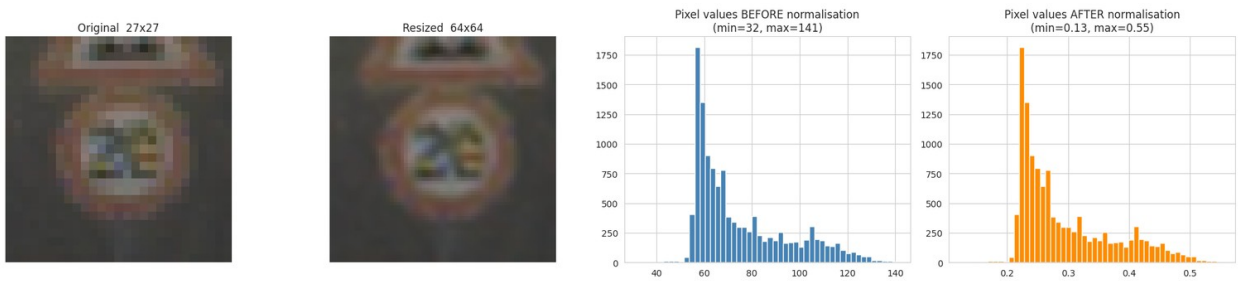


Figure 6.1: Preprocessing check on one image — original, resized, and pixel-value histograms before and after normalisation.

6.5 Resulting Data Tensor

Property	Value
Final image shape	(64, 64, 3)
Full working-set tensor shape (X)	(15460, 64, 64, 3)
Full label vector shape (y)	(15460,)
Stored data type	uint8 (pixel values 0 to 255)
Value range after the Rescaling layer	0.0 to 1.0 (float32, computed inside the model)
Memory footprint of X	≈ 190 MB (a float32 copy would need ≈ 760 MB)

Table 6.1: Final preprocessed data tensor dimensions and memory footprint.

7. Train / Validation / Test Split & Data Preparation

With every image resized into a single, consistent tensor, the next step is to divide the working set into three separate, purpose-specific sets.

7.1 Split Ratios and Purpose

1. **Training set (70%)** — the images the CNN actually learns its filter weights and biases from.
2. **Validation set (15%)** — used during development to monitor training progress, trigger early stopping and choose the best hyperparameters (Section 10), without ever touching the test set.
3. **Test set (15%)** — kept completely untouched until the very end of the project and used only once, to obtain a final, unbiased estimate of how the selected model performs on unseen images.

7.2 Stratified Splitting

Because the classes are not equally sized, a plain random split could, by chance, under-represent a small class such as 0, 19 or 37 in one of the three sets. To prevent this, the split is stratified: each set preserves, as closely as possible, the same relative proportion of each of the 43 classes as the working set (Figure 7.1). The split is performed on indices so that images, labels and file paths always stay aligned, and the original file paths of the test images are retained so that any individual test image can be traced back to its file — a capability used in Section 13.4 to display the misclassified images.

Split	Number of Images	Approximate Proportion
Training	10,822	70.0%
Validation	2,319	15.0%
Test	2,319	15.0%
Total	15,460	100.0%

Table 7.1: Final stratified train / validation / test split sizes.

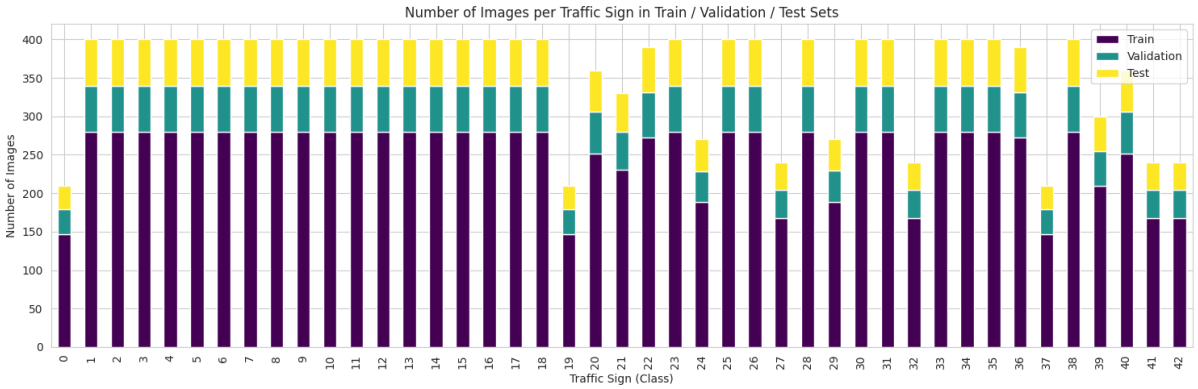


Figure 7.1: Number of images per class in the Train / Validation / Test sets (stratified).

7.3 Class Weights

In addition to stratified splitting, balanced class weights are computed from the training labels only, so that mistakes on smaller classes are penalised more heavily during training than mistakes on larger classes. A weight above 1.0 marks an under-represented class that receives extra importance; a weight below 1.0 marks a class that is better represented. Table 7.2 shows the computed weights, grouped by value.

Class Weight	Classes	Images per Class (subset)
1.71	0, 19, 37	210
1.50	27, 32, 41, 42	240
1.33	24, 29	270
1.20	39	300
1.09	21	330
1.00	20, 40	360
0.92	22, 36	390
0.90	All remaining 28 classes	400

Table 7.2: Class weights used during training (all 43 classes; classes with identical weights grouped).

These weights are passed into the class_weight argument of every model's fit() call, ensuring the loss function accounts for the remaining imbalance, in combination with the stratified splitting strategy above.

8. Data Augmentation

With only about 10,822 training images spread across 43 classes (roughly 250 per class), a CNN can quite easily memorise the specific training photographs rather than learning the general appearance of each sign — a failure mode known as overfitting. Data augmentation counters this by showing the network a slightly modified version of each training image at every epoch, rather than the exact same pixels every time. The random changes chosen mimic real variation in how a camera sees a traffic sign from a moving vehicle.

8.1 Augmentation Techniques Used

1. **Rotation** of up to about ±11° (factor 0.03), simulating a camera or sign that is slightly tilted.
2. **Translation (shift)** of up to 10% of the image width and height, simulating a sign that is not perfectly centred in the frame.
3. **Zoom** of up to 10%, simulating a sign closer to, or further from, the camera.
4. **Contrast change** of up to 20%, simulating signs seen in bright sunlight, shade or at dusk.

8.2 A Deliberate Omission: No Horizontal Flipping

Horizontal flipping is a very common default augmentation in general-purpose image classification, but it is deliberately excluded from this project. Many traffic signs have a meaning that depends on direction: “Turn left ahead” and “Turn right ahead”, “Keep left” and “Keep right”, and “Dangerous curve to the left” and “to the right” are mirror images of each other. Flipping an image would therefore turn one class into another and teach the model something incorrect. This is a clear illustration of why augmentation choices must be grounded in domain understanding rather than applied as a generic, one-size-fits-all default.

8.3 Implementation as In-Model Layers

Rather than augmenting images in a separate offline pass, the transformations are implemented with Keras preprocessing layers (RandomRotation, RandomTranslation, RandomZoom, RandomContrast) placed directly at the start of the model architecture. These layers are active only while the model is in training mode and are automatically switched off during validation, testing and prediction. Every reported validation and test metric is therefore computed on the original, unaltered images, keeping the evaluation fair and interpretable. Figure 8.1 shows one original training image and seven random augmented versions of it.

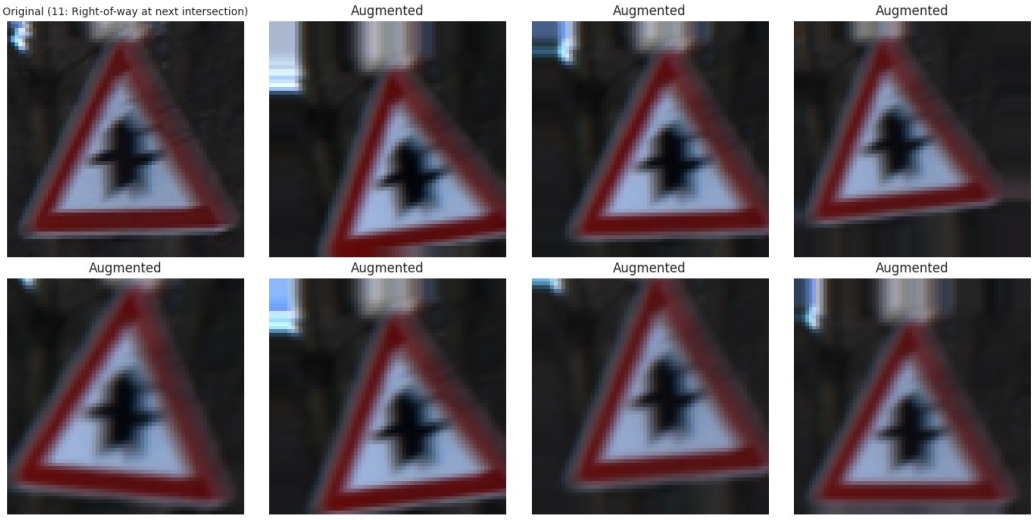


Figure 8.1: One original training image (sign 11) and seven random augmented versions.

9. Model Building & Training — Baseline CNN

A baseline model is deliberately built and trained first, before any hyperparameter tuning is attempted. Its purpose is to establish a fair reference score: if the later, systematically tuned model cannot outperform this reasonably configured baseline, the tuning added no real value. Reporting both models side by side (Section 12) lets the benefit of tuning be judged from evidence rather than assumption.

9.1 How the CNN Works, Layer by Layer

1. **Augmentation + Rescaling** — random augmentation (training only) is applied, then pixel values are divided by 255 (normalisation, Section 6).
2. **Convolution layers (Conv2D)** — small 3×3 filters are slid across the image. The first layer detects simple features such as edges and lines; deeper layers combine them into shapes (circle, triangle, octagon) and finally symbols and numbers.
3. **Max pooling (MaxPooling2D)** — shrinks the feature maps while keeping the strongest responses, making the model faster and less sensitive to the exact position of a feature.
4. **Flatten** — converts the final 2-D feature maps into a single one-dimensional vector.
5. **Dense layer + Dropout** — the flattened features are combined by a fully connected layer to make the classification decision; Dropout randomly deactivates a fraction of neurons during training only, reducing overfitting.
6. **Softmax output layer** — produces one probability for each of the 43 traffic sign classes, summing to 1.0. The class with the highest probability is the prediction.

9.2 Baseline Architecture and Configuration

Layer	Output Shape	Parameters
Input + data_augmentation (training only)	(64, 64, 3)	0
Rescaling (1/255)	(64, 64, 3)	0
Conv2D (32 filters, 3×3, ReLU)	(62, 62, 32)	896
MaxPooling2D (2×2)	(31, 31, 32)	0
Conv2D (64 filters, 3×3, ReLU)	(29, 29, 64)	18,496
MaxPooling2D (2×2)	(14, 14, 64)	0
Conv2D (128 filters, 3×3, ReLU)	(12, 12, 128)	73,856
MaxPooling2D (2×2)	(6, 6, 128)	0
Flatten	(4608)	0
Dense (128, ReLU)	(128)	589,952
Dropout (0.5)	(128)	0
Dense (43, Softmax)	(43)	5,547
Total (all trainable)		688,747

Table 9.1: Baseline CNN layer structure (as reported by the Keras model summary).

Training Property	Value
Total / trainable / non-trainable parameters	688,747 / 688,747 / 0 (2.63 MB)
Optimiser	Adam (learning rate 0.001)

Training Property	Value
Loss function	Sparse categorical cross-entropy
Batch size	32
Maximum epochs	30 (all 30 were run)
Class weights applied	Yes (Section 7.3)
Callbacks	EarlyStopping (patience 5, restores best weights), ReduceLROnPlateau (factor 0.5, patience 2), ModelCheckpoint

Table 9.2: Baseline CNN training configuration.

The model is compiled with the Adam optimiser and sparse categorical cross-entropy loss — "sparse" because the labels are plain integers from 0 to 42 rather than one-hot vectors, a more memory-efficient representation for a 43-class problem. EarlyStopping halts training once validation loss stops improving and restores the best-performing weights, while ReduceLROnPlateau lowers the learning rate when progress stalls, allowing the optimiser to take smaller, more careful steps as it approaches a good solution.

9.3 Baseline Training Behaviour and Results

Training progressed smoothly over all 30 epochs. Training accuracy rose from about 10.0% after the first epoch to about 95.0% by epoch 30, while validation accuracy rose from 36.7% to 99.1% — passing 90% at epoch 7 and 95% at epoch 13. Validation accuracy staying above training accuracy is expected here, because dropout and data augmentation are active only during training, making the training metric a harder, noisier measurement. The learning rate was halved from 0.001 to 0.0005 at epoch 21. The weights from the epoch with the lowest validation loss were restored for evaluation, giving the results in Table 9.3.

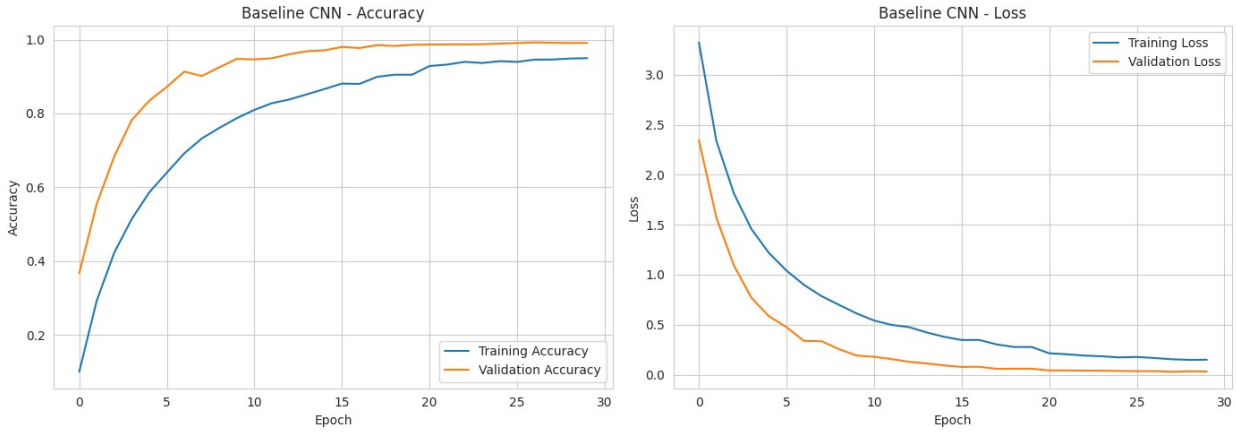


Figure 9.1: Baseline CNN — accuracy and loss curves.

Metric	Value
Baseline CNN — Validation Accuracy	0.9914
Baseline CNN — Test Accuracy	0.9935

Table 9.3: Baseline CNN validation and test accuracy.

A baseline test accuracy of 99.35% is already a strong result for a 43-class problem, and it sets a high bar for the hyperparameter-tuning stage (Section 10) to clear.

10. Hyperparameter Tuning

10.1 Parameters vs. Hyperparameters

A CNN learns its internal weights and biases automatically during training, through backpropagation and gradient descent — these are its parameters. Several structural and training-process choices must, however, be fixed before training begins, such as how many filters a convolutional layer should have, how wide a dense layer should be, what dropout rate and starting learning rate to use, and what batch size to train with. These are hyperparameters, and they are not learned by gradient descent. Rather than guessing them (as the baseline effectively did), this project searches for better values, selecting the combination that performs best on the validation set. The test set is deliberately never used for this selection, so it remains a fair, unbiased judge of the final model.

10.2 Search Space

Six hyperparameters were included in the search space, summarised in Table 10.1. The third convolution layer was kept fixed at 128 filters.

Hyperparameter	Values Tried
Filters in the 1st convolution layer	32, 64
Filters in the 2nd convolution layer	64, 128
Dense layer units	128, 256
Dropout rate	0.3, 0.4, 0.5
Learning rate	0.001, 0.0001
Batch size	16, 32

Table 10.1: Hyperparameter search space (a full grid would need 2×2×2×3×2×2 = 96 combinations).

Exhaustively training all 96 combinations would be prohibitively slow. Keras Tuner's RandomSearch algorithm was therefore used: it randomly samples a fixed number of combinations (trials), trains each candidate briefly (at most 15 epochs, with early stopping of patience 3), and ranks all trials by validation accuracy. The same augmentation block and class weights used for the baseline are applied in every trial, so the later comparison with the baseline remains fair and like-for-like.

10.3 Tuning Results

All 8 trials completed successfully, taking a total of about 2 hours 26 minutes on the available CPU-only hardware. The full ranked results are shown in Table 10.2.

Rank	Batch Size	Filters-1	Filters-2	Dense Units	Dropout	Learning Rate	Val. Accuracy
1	16	32	128	128	0.4	0.0010	0.9918
2	32	32	64	256	0.3	0.0010	0.9897
3	16	32	128	256	0.5	0.0010	0.9871
4	32	32	128	128	0.4	0.0010	0.9823
5	32	32	64	128	0.5	0.0010	0.9702
6	16	32	64	256	0.4	0.0001	0.9556
7	16	32	128	128	0.4	0.0001	0.9495
8	32	32	64	128	0.5	0.0001	0.9064

Table 10.2: All 8 tuning trials, ranked by validation accuracy (highest first).

Two patterns stand out. First, the three weakest trials all used a learning rate of 0.0001, which was too slow to converge within the short per-trial epoch budget; a learning rate of 0.001 dominated the top five ranks. Second, all eight randomly sampled trials happened to use 32 filters in the first convolution layer, so the effect of 64 filters in that layer was not explored in this particular search.

10.4 Best Hyperparameters Selected

Trial 1 achieved the highest validation accuracy (0.9918), and its hyperparameters were therefore selected to build the final CNN in Section 11. These values come directly from this notebook's own tuning run on this dataset, rather than being a pre-decided, generic answer.

Hyperparameter	Selected Value
Filters (convolution layer 1)	32
Filters (convolution layer 2)	128
Dense layer units	128
Dropout rate	0.4
Learning rate	0.001
Batch size	16

Table 10.3: Best hyperparameters selected by Keras Tuner (on validation accuracy).

11. Final CNN Model (Tuned)

The RandomSearch tuner trained each of its 8 candidates only briefly, purely to rank them. The final model is therefore rebuilt from scratch using the best hyperparameters (Table 10.3) and trained again, for longer, on the full training set — still monitored by EarlyStopping (patience 6) and ReduceLROnPlateau, with a higher maximum epoch budget of 40 (compared with 30 for the baseline).

Architecture / Training Property	Value
Filters (convolution layers 1 / 2 / 3)	32 / 128 / 128
Dense layer units	128
Dropout rate	0.4
Learning rate (initial)	0.001
Batch size	16
Total / trainable / non-trainable parameters	780,971 / 780,971 / 0 (2.98 MB)
Maximum epochs	40 (all 40 were run)

Table 11.1: Final tuned CNN architecture and training configuration.

Compared with the baseline, the tuner kept the first-layer filters (32), the dense width (128) and the starting learning rate (0.001), but widened the second convolution layer from 64 to 128 filters, lowered the dropout rate from 0.5 to 0.4 and halved the batch size from 32 to 16. These changes raise the total parameter count from 688,747 to 780,971 (about 13% more), driven by the wider second convolution layer, which also doubles the number of input channels reaching the third.

Training progressed rapidly and smoothly. Training accuracy rose from 18.8% after epoch 1 to 99.0% by epoch 40, while validation accuracy rose from 49.4% to above 95% by epoch 7, above 98% by epoch 10 and above 99% by epoch 16, peaking at 99.74%. Validation loss fell steadily and consistently throughout, and the learning rate was stepped down automatically from 0.001 to 0.0005 (epoch 16), 0.00025 (epoch 28), 0.000125 (epoch 32) and 0.0000625 (epoch 39). Each epoch took roughly 77–82 seconds on CPU. The best-validation-loss weights were restored for evaluation. The training curves (Figure 11.1) show no sign of erratic overfitting.

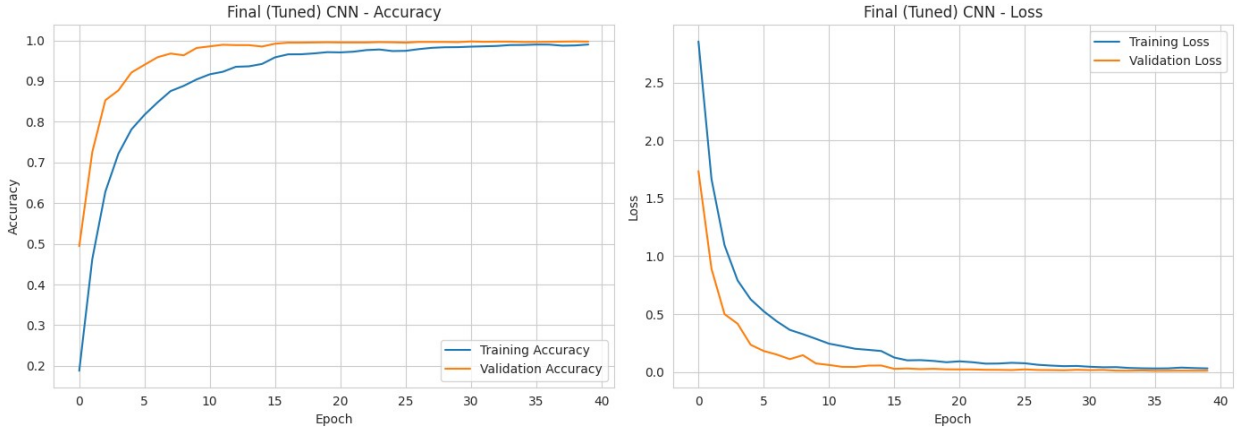


Figure 11.1: Final (tuned) CNN — accuracy and loss curves.

Metric	Value
Final Tuned CNN — Validation Accuracy	0.9961
Final Tuned CNN — Test Accuracy	0.9953

Table 11.2: Final tuned CNN validation and test accuracy.

The final tuned CNN improves on the baseline on both validation accuracy (99.61% versus 99.14%) and test accuracy (99.53% versus 99.35%). These results are discussed in full in Sections 12 and 13.

12. Model Comparison Report

The baseline CNN (Section 9) and the tuned final CNN (Section 11) are compared side by side on both the validation set and the test set, using overall Accuracy together with macro-averaged Precision, Recall and F1-score. Macro averaging computes each metric independently for each of the 43 classes and then averages the results with equal weight, regardless of how many images each class has — the appropriate choice for a multi-class dataset with unequal class sizes, since it prevents large classes from dominating the headline score at the expense of small ones.

Model / Split	Accuracy	Precision (macro)	Recall (macro)	F1-Score (macro)
Baseline CNN — Validation	0.9914	0.9920	0.9918	0.9918
Baseline CNN — Test	0.9935	0.9941	0.9939	0.9940
Final Tuned CNN — Validation	0.9961	0.9964	0.9960	0.9961
Final Tuned CNN — Test	0.9953	0.9957	0.9957	0.9957

Table 12.1: Baseline vs. Final Tuned CNN, compared on validation and test splits.



Figure 12.1: Baseline vs. Final Tuned CNN on the test set (Accuracy, macro Precision, Recall and F1-score).

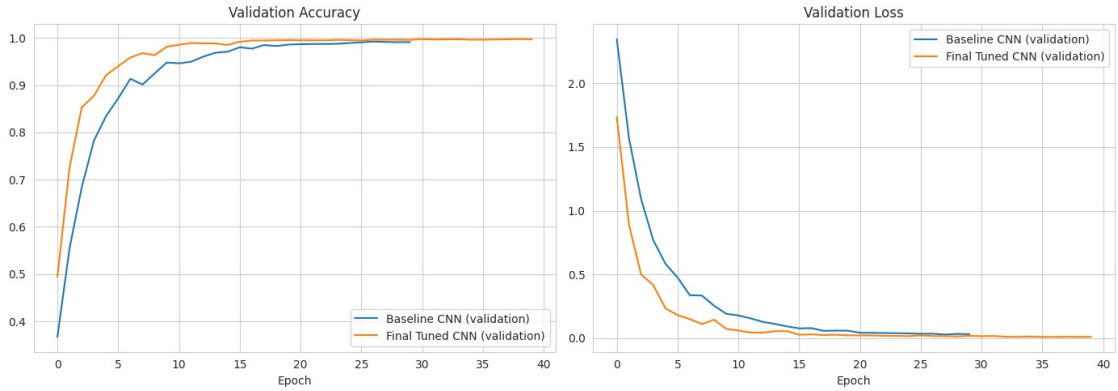


Figure 12.2: Validation accuracy and validation loss of both models across training epochs.

12.1 Selecting the Best Model

Consistent with sound machine learning practice, the best model is selected using validation accuracy, not test accuracy. The Final Tuned CNN's validation accuracy of 0.9961 exceeds the Baseline CNN's 0.9914, so the Final Tuned CNN is selected as the best model. Crucially, this decision is made without looking at either model's test-set performance, so the test scores reported afterwards remain an honest, unbiased estimate of how the selected model performs on images it never influenced any decision about.

On the test set, the selected model achieves an accuracy of 0.9953 (99.53%), with macro Precision, Recall and F1-score all equal to 0.9957 — meaning it correctly classified 2,308 of the 2,319 test images.

12.2 Did Tuning Help?

Because both models were evaluated identically, the benefit of tuning can be judged directly from Table 12.1. Test accuracy improved from 99.35% to 99.53% and macro F1-score from 0.9940 to 0.9957; validation accuracy improved from 99.14% to 99.61%. The baseline was already highly accurate, so the improvement is modest — on the order of four additional correctly classified test images out of 2,319 — and, with a single training run per model, differences of this size should be read as a small but consistent gain on both splits rather than as a large effect. The tuning stage nevertheless delivered a measurable, evidence-backed improvement, and it also provided useful knowledge about the model (for example, that a learning rate of 0.001 clearly outperforms 0.0001 for this problem and training budget).

13. Best Model — Detailed Evaluation

The selected Final Tuned CNN is evaluated in further depth on the untouched test set, using a full per-class classification report, a confusion matrix, per-sign F1-score analysis, an analysis of the most frequently confused sign pairs and a review of the misclassified images.

13.1 Classification Report

Table 13.1 gives the complete classification report for the Final Tuned CNN on all 2,319 test images. Precision answers “of the images predicted as this sign, how many really were it?”, recall answers “of all real images of this sign, how many were found?”, F1-score balances the two, and support is the number of test images of that class. Every class achieves precision and recall of at least 0.96, and the overall accuracy is 0.9953.

Class	Precision	Recall	F1-Score	Support
0: Speed limit (20km/h)	1.00	1.00	1.00	31
1: Speed limit (30km/h)	0.98	1.00	0.99	60
2: Speed limit (50km/h)	1.00	0.98	0.99	60
3: Speed limit (60km/h)	0.97	1.00	0.98	60
4: Speed limit (70km/h)	1.00	0.98	0.99	60
5: Speed limit (80km/h)	0.98	0.97	0.97	60
6: End of speed limit (80km/h)	1.00	1.00	1.00	60
7: Speed limit (100km/h)	0.98	0.98	0.98	60
8: Speed limit (120km/h)	1.00	1.00	1.00	60
9: No passing	1.00	1.00	1.00	60
10: No passing for vehicles over 3.5 tons	1.00	1.00	1.00	60
11: Right-of-way at next intersection	1.00	0.97	0.98	60
12: Priority road	1.00	1.00	1.00	60
13: Yield	1.00	1.00	1.00	60
14: Stop	1.00	1.00	1.00	60
15: No vehicles	1.00	1.00	1.00	60
16: Vehicles over 3.5 tons prohibited	1.00	1.00	1.00	60
17: No entry	1.00	1.00	1.00	60
18: General caution	1.00	0.98	0.99	60
19: Dangerous curve to the left	1.00	1.00	1.00	31
20: Dangerous curve to the right	0.96	1.00	0.98	54
21: Double curve	1.00	1.00	1.00	50
22: Bumpy road	1.00	1.00	1.00	59
23: Slippery road	1.00	1.00	1.00	60
24: Road narrows on the right	1.00	1.00	1.00	41
25: Road work	0.97	1.00	0.98	60
26: Traffic signals	1.00	1.00	1.00	60
27: Pedestrians	1.00	1.00	1.00	36

Class	Precision	Recall	F1-Score	Support
28: Children crossing	1.00	0.97	0.98	60
29: Bicycles crossing	1.00	1.00	1.00	40
30: Beware of ice/snow	0.98	1.00	0.99	60
31: Wild animals crossing	1.00	1.00	1.00	60
32: End of all speed and passing limits	1.00	1.00	1.00	36
33: Turn right ahead	1.00	0.98	0.99	60
34: Turn left ahead	1.00	1.00	1.00	60
35: Ahead only	1.00	1.00	1.00	60
36: Go straight or right	1.00	1.00	1.00	59
37: Go straight or left	1.00	1.00	1.00	31
38: Keep right	1.00	1.00	1.00	60
39: Keep left	1.00	1.00	1.00	45
40: Roundabout mandatory	0.98	1.00	0.99	54
41: End of no passing	1.00	1.00	1.00	36
42: End of no passing by vehicles over 3.5 tons	1.00	1.00	1.00	36
Overall Accuracy			0.9953	2319

Table 13.1: Per-class classification report for the Final Tuned CNN on the test set (macro-average Precision, Recall and F1-score: 0.9957 each).

13.2 Confusion Matrix

The 43×43 confusion matrix for the test set (Figure 13.1) is almost perfectly diagonal: rows are the actual signs, columns are the predicted signs, and correct predictions lie on the diagonal. Only a handful of off-diagonal cells contain a non-zero value, each of them a small count of one or two images, showing that the model confuses very few sign pairs.

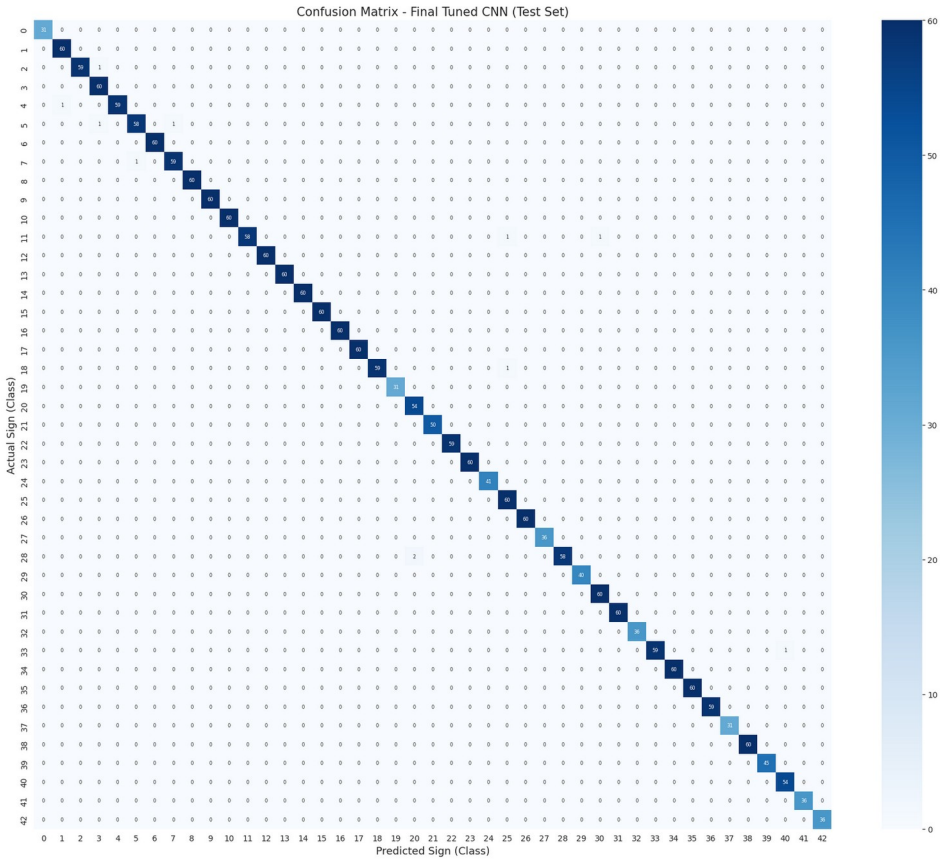


Figure 13.1: Confusion matrix of the Final Tuned CNN on the test set.

13.3 Per-Sign F1-Scores and Most-Confused Pairs

Figure 13.2 plots the F1-score of every sign, sorted from weakest to strongest. The five weakest signs are 5 — Speed limit (80 km/h) (0.97), 20 — Dangerous curve to the right (0.98), 28 — Children crossing (0.98), 11 — Right-of-way at next intersection (0.98) and 7 — Speed limit (100 km/h) (0.98). The strongest signs, all at 1.00, include 38 — Keep right, 35 — Ahead only, 39 — Keep left, 41 — End of no passing and 42 — End of no passing by vehicles over 3.5 tons. Even the weakest sign therefore reaches an F1-score of 0.97.

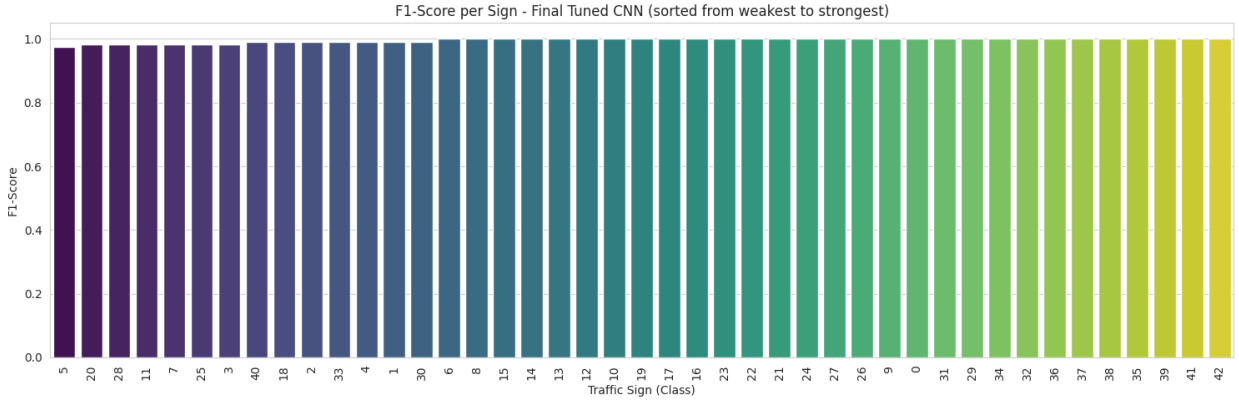


Figure 13.2: F1-score per sign for the Final Tuned CNN, sorted from weakest to strongest.

Table 13.2 lists the most frequently confused pairs of signs, i.e. the largest off-diagonal values of the confusion matrix. The only pair confused more than once is Children crossing predicted as Dangerous curve to the right (2 images); all others occurred once. As anticipated in the domain analysis, several of the errors are among the speed-limit signs (50→60, 70→30, 80→60, 100→80 and 80→100 km/h), which differ only in the number inside the circle, while others involve red-bordered warning triangles that look alike at 64×64 resolution (Right-of-way, General caution and Road work).

Actual Sign	Predicted As	Images
28: Children crossing	20: Dangerous curve to the right	2
2: Speed limit (50km/h)	3: Speed limit (60km/h)	1
4: Speed limit (70km/h)	1: Speed limit (30km/h)	1
5: Speed limit (80km/h)	3: Speed limit (60km/h)	1
7: Speed limit (100km/h)	5: Speed limit (80km/h)	1
5: Speed limit (80km/h)	7: Speed limit (100km/h)	1
11: Right-of-way at next intersection	25: Road work	1
11: Right-of-way at next intersection	30: Beware of ice/snow	1
18: General caution	25: Road work	1
33: Turn right ahead	40: Roundabout mandatory	1

Table 13.2: The ten most frequently confused sign pairs on the test set (11 misclassified images in total).

13.4 Misclassified Images

A dedicated review step collected the misclassified test images: 11 out of 2,319 (0.47%). Figure 13.3 shows them using the original, un-resized photographs, with the actual sign, the predicted sign and the model's confidence for each. Many of these photographs are visibly small, dark, blurred or partly obscured, which helps explain why the model was misled.



Figure 13.3: The misclassified test images (actual sign, predicted sign and model confidence).

13.5 Interpreting the Results

A test accuracy of 99.53% is an excellent outcome on paper, but it should be interpreted with appropriate scientific caution, and this report does so openly in Section 20. Three observations are relevant. First, the dataset has 43 well-defined, visually distinctive classes and a clean separation between shape/colour

categories, so a moderately deep CNN can learn the task very well. Second, the image file names in the dataset follow a class_sequence_frame pattern (for example 00000_00000_00000.png), which is consistent with the GTSRB benchmark, where each physical sign is photographed in a sequence of near-identical consecutive frames. If that is the case here, a random, image-level stratified split will place different frames of the same physical sign in the training, validation and test sets, and the test set will then be more similar to the training set than a truly independent set of new signs would be. Third, only 11 errors occurred, so the per-class figures rest on small counts (at most 60 test images per class), and a single image changes a class F1-score by about 0.01.

The result is therefore best read as strong evidence that the model fits this dataset extremely well, rather than as proof that it would perform equally well on signs from new roads, new cameras, other countries or harsher weather. The external-validation and group-aware-split recommendations in Section 21 address exactly this point.

14. Predicting the Traffic Sign of a New Image

This section demonstrates the complete, end-to-end application working as a whole, from a raw input image to a final, human-readable prediction:

New traffic sign image → Resize (64×64) → CNN (normalisation inside) → Probability for each of the 43 signs → Highest probability → Predicted traffic sign

The predict_sign() function reuses the same preprocess_image() function introduced in Section 6, which was also used to prepare every training image. This guarantees that a brand-new photograph is treated exactly like the images the model was trained on, removing a common, subtle source of deployment errors where training-time and inference-time preprocessing accidentally diverge.

The function returns three pieces of information: the single predicted sign (the class with the highest softmax probability), the model's confidence for that prediction (the raw softmax probability, expressed as a percentage), and the top-3 most likely signs with their individual probabilities, which gives visibility into cases where the model may be torn between two visually similar signs. As the notebook itself stresses, confidence is the model's own estimate of certainty — it is not a guarantee of correctness, and a confident but wrong prediction remains possible on unusual input images.

In the demonstration run, four randomly sampled images from the unseen test set were passed through predict_sign(). All four were classified correctly, as summarised in Table 14.1 and shown in Figures 14.1 to 14.4.

Actual Sign	Predicted Sign	Confidence	Result
29: Bicycles crossing	29: Bicycles crossing	100.0%	CORRECT
21: Double curve	21: Double curve	99.9%	CORRECT
28: Children crossing	28: Children crossing	100.0%	CORRECT
11: Right-of-way at next intersection	11: Right-of-way at next intersection	100.0%	CORRECT

Table 14.1: End-to-end prediction demonstration on four random test images.

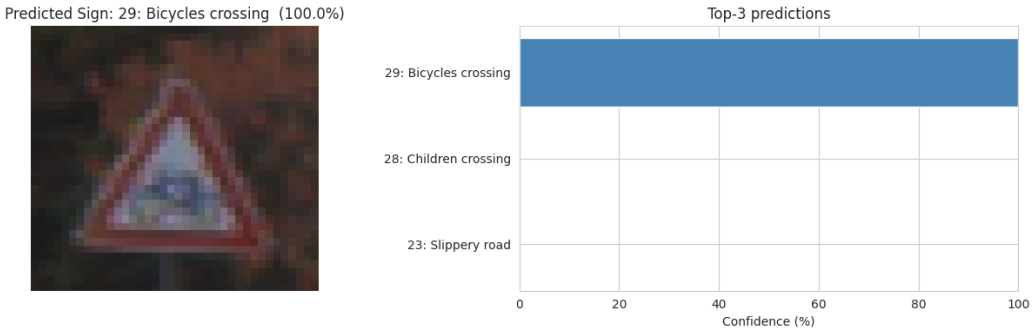


Figure 14.1: Prediction for a Bicycles crossing sign, with top-3 probabilities.

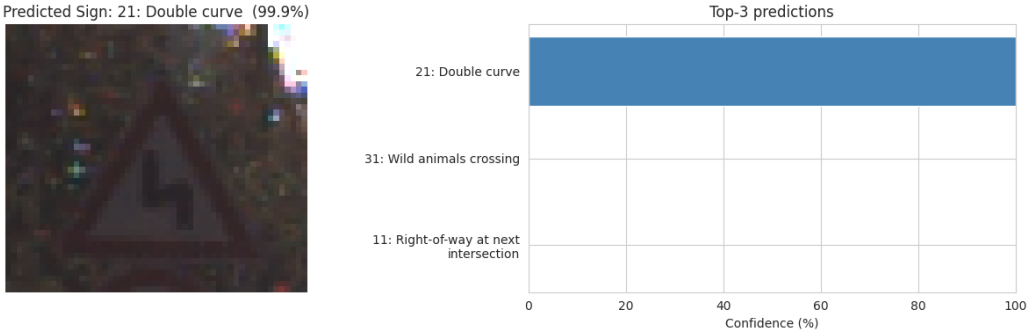


Figure 14.2: Prediction for a Double curve sign, with top-3 probabilities.

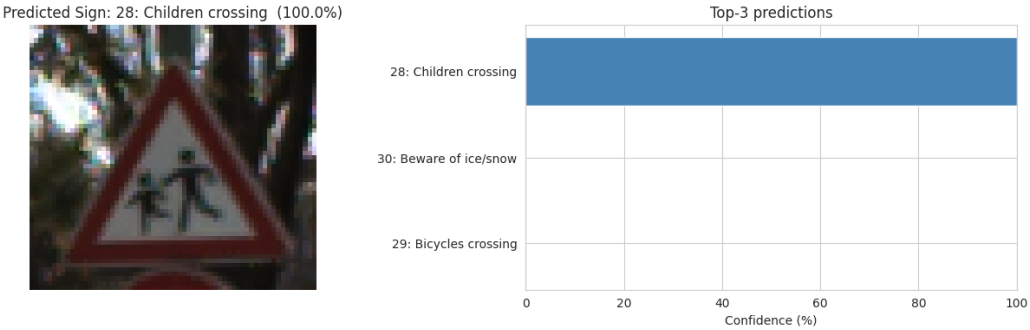


Figure 14.3: Prediction for a Children crossing sign, with top-3 probabilities.

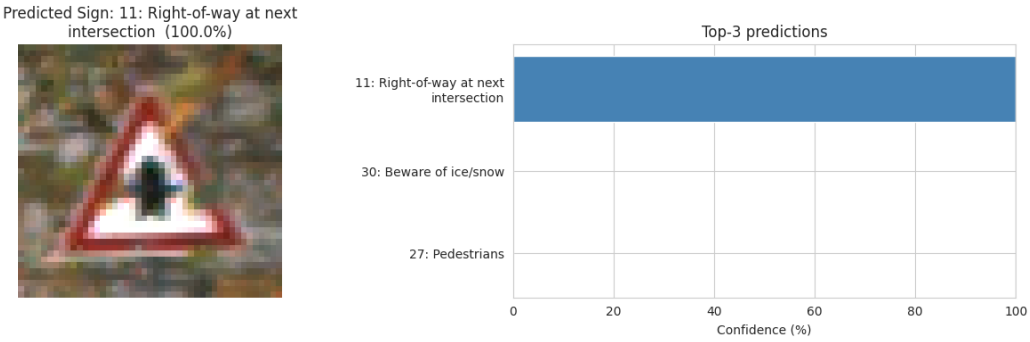


Figure 14.4: Prediction for a Right-of-way at next intersection sign, with top-3 probabilities.

The notebook also contains an optional cell that lets a user predict the sign of their own image, either by giving a file path or, in Google Colab, through an upload button.

15. Saving the Model

The selected Final Tuned CNN is saved to disk in the native Keras format (a single .keras file), together with a separate JSON file containing the class folder names, the readable sign names and the image size used in preprocessing. Saving the class-name mapping alongside the model is essential: a raw model file only outputs numeric class indices (0 to 42), and without the corresponding mapping those numbers would be meaningless to any future application that loads the model.

Artefact	Detail
Saved model file	TrafficSign_CNN_Model.keras (≈ 9.4 MB)
Saved class names file	TrafficSign_class_names.json (class folders, readable sign names, image size 64)
Reload verification	Reloaded model's predictions on 20 test images confirmed identical to the original in-memory model (True)

Table 15.1: Saved model artefacts and verification result.

As a final integrity check, the saved model was reloaded from disk and used to regenerate predictions, which were compared against the original model's predictions. The two were found to be identical, confirming that the save/reload round-trip preserves the model exactly. This gives confidence that the .keras file, together with its class-names JSON, is ready to be consumed by a separate application — for example a web or mobile app — without retraining. At about 9.4 MB, the model is also small enough to be a realistic candidate for deployment on modest hardware.

15.1 Final Project Results Summary

Metric	Value
Number of traffic sign classes	43
Images (Train / Validation / Test)	10,822 / 2,319 / 2,319
Input size	64 × 64 × 3
Selected model	Final Tuned CNN
Test accuracy	99.53%
Test precision (macro)	0.9957
Test recall (macro)	0.9957
Test F1-score (macro)	0.9957

Table 15.2: Final, official project results summary, as printed at the end of the notebook.

16. Business Insights

16.1 Road Safety and Driver Assistance

The trained CNN turns a photograph of a traffic sign into a predicted sign class within a fraction of a second. Integrated into a driver-assistance system, a model of this kind can alert drivers to speed limits, stop signs and hazard warnings they may have missed, helping to reduce accidents caused by missed or misread signs — precisely the problem identified in Section 1.1.

16.2 Foundation for Autonomous Vehicles

Self-driving and semi-autonomous vehicles must read traffic signs to obey road rules. A reliable sign classifier is one of the building blocks of such systems, and the preprocessing and evaluation pipeline built here can be reused when more data or a detection stage is added.

16.3 Preprocessing Makes Real-World Use Possible

The raw photographs came in 2,066 different resolutions and quality levels (Section 4.4). Resizing every image — and, critically, applying exactly the same steps to new images at prediction time (Section 14) — is what allows a single trained model to handle photographs from different cameras consistently, a prerequisite for any real deployment where images come from whatever sensor a vehicle happens to carry.

16.4 Confusion Analysis Shows Where to Improve

The confusion matrix, the per-sign F1-scores and the misclassified-image gallery (Section 13) show which signs are confused with one another. Signs that look alike — for example different speed limits — are the best targets for collecting more varied training photographs, so that improvement effort is focused rather than spread across the whole dataset. In a safety setting, confusing two speed limits can matter more than confusing two unrelated signs, so these errors deserve special attention.

16.5 Hyperparameter Tuning Adds Measurable Value

The tuned model was compared against the untuned baseline on the same validation and test data (Section 12): test accuracy improved from 99.35% to 99.53% and macro F1-score from 0.9940 to 0.9957. Because the comparison is reported openly, the benefit of the roughly 2.5 hours invested in hyperparameter search can be judged from evidence rather than assumption. The gain is modest because the baseline was already strong, which is itself a useful finding: for a problem like this, a sensible baseline captures most of the achievable accuracy, and tuning effort is best weighed against the cost of compute.

16.6 Scope and Deployment Considerations

The model classifies a sign that has already been cropped out of a photograph. A real system must also detect where signs are in a full road scene, work in rain, night and glare, and be validated thoroughly before it is relied on for any safety-critical decision (see Sections 20 and 21).

17. Challenges Faced

17.1 Dataset Too Large to Load at Once

The full dataset contains 39,209 images, which is too much to extract and hold in memory as arrays on normal hardware. This was solved by (a) reading the ZIP index without extracting it, (b) extracting only a fixed number of randomly chosen images per class (MAX_IMAGES_PER_CLASS = 400), (c) resizing to 64×64 and (d) keeping the arrays as uint8 and normalising inside the model, which cut the memory footprint of the image array from about 760 MB to 190 MB. The cap can be raised or removed if more memory is available.

17.2 Images of Many Different Sizes and Quality

The photographs vary widely in resolution (2,066 distinct sizes) and many are small, blurred, dark or bright. All were converted to a common 64×64 RGB format, and contrast augmentation was added to make the model robust to lighting changes.

17.3 Class Imbalance

Some signs have more than ten times as many images as others in the full dataset (10.71 : 1). This was handled by capping images per class (reducing the ratio to 1.90 : 1), stratified train/validation/test splitting and class weights during training, and the models are judged with macro-averaged metrics so that small classes count equally.

17.4 Visually Similar and Mirror-Image Signs

Certain signs differ only by a small detail (such as the number on a speed-limit sign), and some signs are mirror images of each other (turn left / turn right). Horizontal flipping was therefore deliberately avoided in augmentation, and the confusion matrix and classification report were examined sign by sign instead of relying only on overall accuracy. The remaining 11 test errors are indeed concentrated among look-alike signs.

17.5 Risk of Overfitting

A CNN can memorise the training photographs. Data augmentation (rotation, shift, zoom, contrast), dropout, early stopping and learning-rate reduction were all used together to reduce this risk (Sections 8 and 9).

17.6 Avoiding Information Leakage in Model Selection

To keep the final reported score honest, both the hyperparameter search (Section 10) and the choice between the baseline and tuned model (Section 12.1) were made using only the validation set. The test set was used only once, at the end, for the final evaluation reported in Section 13.

17.7 Compute Limits for Hyperparameter Search

Searching many hyperparameter combinations is slow, particularly without a GPU. The eight tuner trials took about 2 hours 26 minutes on CPU. The number of tuner trials and epochs are configurable (TUNER_TRIALS, TUNER_EPOCHS) so the notebook can be run on modest hardware, and a GPU runtime (for example in Google Colab) is recommended.

18. Final Conclusion

18.1 Project Objective

The objective of this project was to build a machine learning model that recognises traffic signs from images, covering both explicit goals of the DataMites brief: image preprocessing, and an ML model to predict the traffic sign.

18.2 Exploratory Data Analysis

EDA showed a 43-class dataset of traffic sign photographs with strongly unequal class sizes (10.71 : 1), 2,066 different image resolutions, and natural variation between images of the same sign in lighting, blur, size and angle — findings that directly justified the memory-safe loading strategy and the resizing, normalisation, stratified splitting, class-weighting and data-augmentation choices.

18.3 Modelling

A baseline CNN was built first, then Keras Tuner's RandomSearch was used to select the number of convolutional filters, dense units, dropout rate, learning rate and batch size, using only validation accuracy as the selection criterion. A final CNN was trained from scratch with the best hyperparameters, and both models were compared fairly using identical metrics on identical data splits.

18.4 Outcome

The selected Final Tuned CNN achieved a test accuracy of 99.53% with macro Precision, Recall and F1-score of 0.9957, correctly classifying 2,308 of 2,319 unseen test images across all 43 classes; the baseline reached 99.35%. The remaining 11 errors are concentrated in visually similar signs such as the speed limits. The model can predict the traffic sign of a new image together with its confidence (Section 14) and has been saved for reuse (Section 15). Sections 13.5 and 20 discuss why this score should be read as strong evidence of a good fit to this dataset rather than a guarantee for all real-world conditions. Future work, outlined in Section 21, includes using the full dataset, a group-aware split and external validation, transfer learning with pretrained networks, a detection stage to locate signs in full road scenes, and real-time camera recognition.

19. Project Links

The following links provide access to the complete project resources, including the source code, dataset and supporting documentation.

19.1 GitHub Repository

Access to the complete source code, Jupyter Notebook, README and project files:

https://github.com/Sameekshagithub/PRAICP-1002-Traffic-Sign-Detection-Using-Deep-Learning.git

19.2 Google Drive

Access to the accompanying presentation (PPT) and this report:

https://drive.google.com/drive/folders/1pcADNaZ2l3gGNx7uDhTEBlcqMi5VIEOh?usp=drive_link

20. Ethical Considerations & Limitations

20.1 Ethical Considerations

1. **Safety-Critical Use:** A wrong or missed sign prediction in a vehicle could contribute to an accident. A production system must never rely on a single model output; it should report its confidence, fall back safely when confidence is low, keep the human driver responsible in driver-assistance settings, and be tested to the standards required for safety-related automotive software.
2. **Dataset Representativeness and Geography:** The class numbering follows the German traffic sign benchmark. Signs differ from country to country in design, colour and meaning, so a model trained on this dataset must not be assumed to read signs correctly in other countries (including India) without retraining and validation on local signs.
3. **Fairness Across Conditions:** Real roads include night, rain, fog, glare, dirt, damaged and partly hidden signs. The dataset contains many dark and blurred images but does not systematically cover all such conditions, and performance may be worse for the conditions that are under-represented.
4. **Robustness and Misuse:** Image classifiers can be fooled by small physical changes, such as stickers or paint on a sign. Safety-related deployments must consider such adversarial risks and add redundancy (for example map data or multiple sensors).
5. **Data Privacy:** Photographs taken from vehicles can capture people, faces and number plates. Any real deployment that stores or shares such images must handle them responsibly, with appropriate consent and data-protection safeguards under applicable regulations such as India's Digital Personal Data Protection Act.

20.2 Limitations of this Project

1. **Optimistic Evaluation Likely:** The train, validation and test sets were created by a random, stratified split of individual images. If, as the file naming suggests, the dataset contains sequences of near-identical frames of the same physical sign, frames of the same sign appear in more than one split, which can make the test score higher than it would be on genuinely new signs (Section 13.5).
2. **Subset of the Data Used:** To keep memory use practical, at most 400 images per class (15,460 of 39,209 images) were used. The large classes were therefore under-used, and the results for the full dataset may differ.
3. **Classification, Not Detection:** The model classifies cropped sign images. It does not locate signs within a full road scene, handle several signs in one image, or process video.
4. **Low Resolution:** Images were reduced to 64×64 pixels. This kept training fast but discards fine detail, and is a likely reason for confusions between look-alike signs such as speed limits.
5. **Single-Dataset, Single-Run Evaluation:** All models were trained and evaluated on one dataset, with one training run per model and a short random search of only eight trials (in which the first layer happened to use 32 filters in every trial). The 0.18-point gain of the tuned model over the baseline on the test set is small and should not be over-interpreted.
6. **No Field Validation:** This project is a technical, academic proof-of-concept. It has not been tested in a vehicle or on live camera input, and should not be treated as a validated, ready-to-deploy driver-assistance product.

21. Future Scope & Recommendations

1. **Group-Aware Splitting and External Validation:** Split the data so that all frames of one physical sign stay in the same set (when sequence information is available), and evaluate the model on a completely independent set of traffic sign photographs, to obtain a realistic estimate of real-world accuracy.
2. **Use the Full Dataset:** Train on all 39,209 images on a GPU-enabled machine (by raising or removing the per-class cap), possibly with a data generator that loads images in batches so memory stays manageable.
3. **Higher Input Resolution:** Test larger input sizes (for example 96×96 or 128×128) to help separate look-alike signs such as the speed limits, and compare the accuracy gain against the extra training cost.
4. **Transfer Learning:** Build on ImageNet-pretrained backbones such as MobileNetV2 or EfficientNet instead of training a CNN entirely from scratch, which may improve robustness and generalisation.
5. **Add a Detection Stage:** Combine the classifier with an object detector (for example YOLO, SSD or Faster R-CNN) so that signs are first located in a full road scene and then classified.
6. **Real-Time Camera Recognition:** Package the saved .keras model (Section 15) into an application that processes live camera frames through the same preprocessing and prediction pipeline used in Section 14, using a lighter format such as TensorFlow Lite for edge devices.
7. **Robustness Testing:** Evaluate and improve performance under night, rain, fog, glare, motion blur and partial occlusion, using targeted augmentation or additional data.
8. **Targeted Error Reduction:** Collect more varied photographs of the most confused signs (the speed limits and red-bordered warning triangles) and consider confidence thresholds or second-stage checks for safety-relevant classes.
9. **Explainability:** Use Grad-CAM or similar techniques to show which regions of a sign image influenced a prediction, helping to build trust and diagnose errors.
10. **Local Sign Sets:** Collect and train on traffic signs from the country of intended deployment (for example Indian road signs) rather than assuming the model transfers across countries.

22. References

1. Stallkamp, J., Schlipsing, M., Salmen, J., & Igel, C. (2012). Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32, 323–332.

2. O'Shea, K., & Nash, R. (2015). An Introduction to Convolutional Neural Networks. *arXiv preprint*.

3. Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.

4. Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations (ICLR)*.

5. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.

6. O'Malley, T., Bursztein, E., Long, J., et al. (2019). KerasTuner. Retrieved from the official Keras Tuner documentation.

7. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

8. TensorFlow Developers. TensorFlow / Keras API Documentation. Retrieved from the official TensorFlow documentation site.

9. DataMites® Innovative Solutions Pvt. Ltd. Artificial Intelligence Capstone Project Guidelines, PRAICP-1002.

Appendix A: Glossary of Terms

CNN (Convolutional Neural Network): A class of deep neural networks that use convolutional layers to automatically learn spatial hierarchies of features from image data.

Traffic Sign Classification: Predicting which traffic sign class an image of a (cropped) sign belongs to, as opposed to detection, which also locates the sign within a larger scene.

GTSRB: German Traffic Sign Recognition Benchmark, a widely used 43-class traffic sign dataset whose class numbering matches this project's folder numbering.

Stratified Split: A way of dividing a dataset into subsets (e.g. train/validation/test) that preserves the same relative proportion of each class in every subset.

Class Weighting: A technique that assigns higher loss penalties to under-represented classes during training, to counter the effects of class imbalance.

Data Augmentation: Applying random, label-preserving transformations (rotation, shifting, zooming, contrast change) to training images to increase data diversity and reduce overfitting.

Rescaling Layer: A Keras layer that multiplies inputs by a constant (here 1/255), normalising pixel values to the range 0–1 inside the model.

Hyperparameter: A configuration value (e.g. learning rate, number of filters, dropout rate) that is set before training begins, rather than learned from data.

Keras Tuner / RandomSearch: A library and algorithm that automates the search for good hyperparameter combinations by randomly sampling and evaluating candidates from a defined search space.

Epoch / Batch Size: An epoch is one full pass through the training data; the batch size is the number of images processed before the model weights are updated.

Softmax: An activation function used in the output layer of a multi-class classifier that converts raw outputs into a probability distribution over all classes, summing to 1.0.

Macro Average: A way of averaging a metric (e.g. F1-score) across classes by giving each class equal weight, regardless of how many samples it has.

Confusion Matrix: A table showing, for each true class, how many samples were predicted as each possible class; the diagonal shows correct predictions and off-diagonal cells show confusions.

Early Stopping: A regularisation technique that halts training once a monitored metric (e.g. validation loss) stops improving, to prevent overfitting.

ReduceLROnPlateau: A callback that lowers the learning rate when a monitored metric stops improving, so the optimiser takes smaller steps near a good solution.

Dropout: A regularisation technique that randomly deactivates a fraction of neurons during training to reduce overfitting and co-dependency between neurons.

Overfitting: When a model memorises its training data and performs noticeably worse on new, unseen data.

Appendix B: Environment & Reproducibility Notes

Component	Detail
Programming language	Python 3
Deep learning framework	TensorFlow 2.21.0 (Keras high-level API)
Hyperparameter tuning library	Keras Tuner 1.4.8 (RandomSearch)
Hardware used	CPU only (no GPU acceleration available in the training environment)
Notebook environment	Jupyter Notebook
Random seed	42 (Python, NumPy, TensorFlow, sampling and splitting)
Dataset size	39,209 images in the ZIP; 15,460 used (≤ 400 per class); ≈ 190 MB as uint8 tensors
Hyperparameter search time	≈ 2 hours 26 minutes across 8 trials (max 15 epochs per trial)
Training time per epoch	≈ 50–85 s (baseline, batch 32); ≈ 77–82 s (final, batch 16)
Model persistence format	Native Keras (.keras, ≈ 9.4 MB) plus a class-names JSON file

The absence of GPU acceleration materially affected training and search time. Readers reproducing this project on GPU-enabled hardware should expect substantially shorter times, though final model metrics should remain broadly consistent given the same data, architecture and augmentation policy, subject to the normal run-to-run variance of stochastic neural network training and of the random sampling used by RandomSearch.

For full reproducibility, refer to the original Jupyter Notebook (PRAICP_1002_TrafSignDetection.ipynb), which contains the complete, executable source code corresponding to every section of this report, including the configurable MAX_IMAGES_PER_CLASS, TUNER_TRIALS and TUNER_EPOCHS parameters referenced in Sections 3.1 and 17. The repository and Drive links are given in Section 19.